Báo cáo thực hành

Sinh viên thực hiện: Lâm Quang Khôi

MSSV: 3122411100 - Lớp: 3122411100

Chủ đề thực hiện: Lab 3 + 4

Lab2: Seq2Seq

1. Mục tiêu

- Hiểu được cách hoạt động và cài đặt mô hình Seq2Seq.
    - Giải thích được vai trò của encoder và decoder.
    - Xây dựng mô hình Seq2Seq dựa trên GRU.
    - Huấn luyện với học thúc đẩy (Teaching forcing).
    - Thực hiện giải mã tham lam ở giai đoạn dự đoán.
    - Đánh giá các bản dịch bằng độ chính xác khớp hoàn toàn đơn giản hoặc điểm số BLEU.

2. Phương pháp

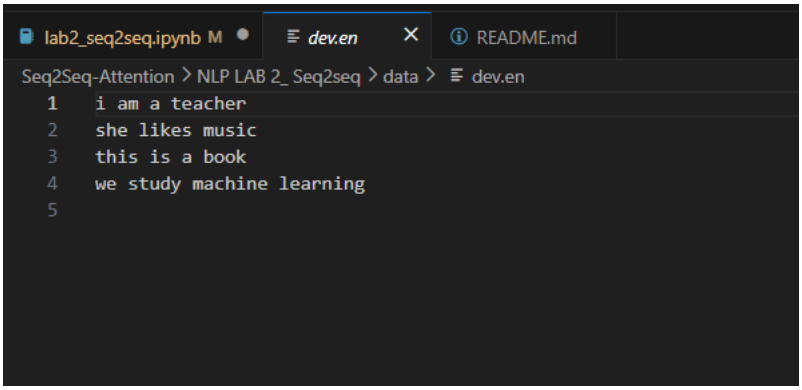
- Thực hiện theo handout của sinh viên:
    - Tiền xử lý dữ liệu (vận dụng từ Lab 1).
    - Mã hóa thứ tự câu (encoding).
    - Giải hóa dữ liệu sau khi huấn luyện (decoding).
    - Huấn luyện mô hình dùng kỹ thuật Teaching Forcing.
    - Chỉnh sửa để giải mã tham lam.
    - Đánh giá thông qua 2 cách: So sánh độ chính xác so với dataset mẫu, hoặc dùng độ đo BLEU khi dùng NLTK.

3. Quá trình thực hiện

Dữ liệu sử dụng

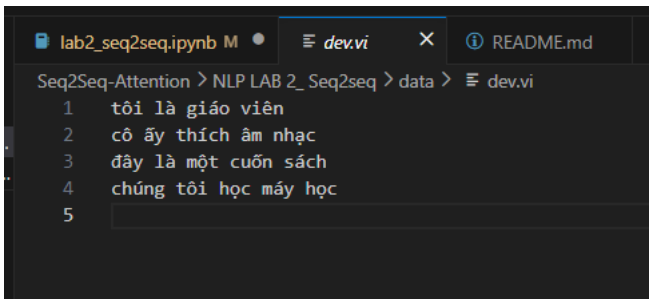
Có 4 bộ dữ liệu chính:

- dev.en: Dữ liệu lập trình mẫu tiếng Anh.



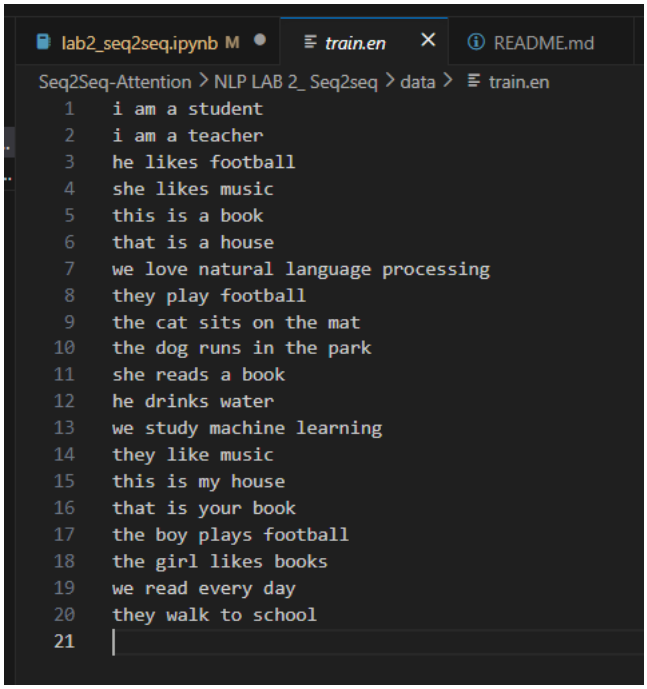
Pic 1. Data dev.en.

- dev.vi: Dữ liệu lập trình mẫu tiếng Việt.



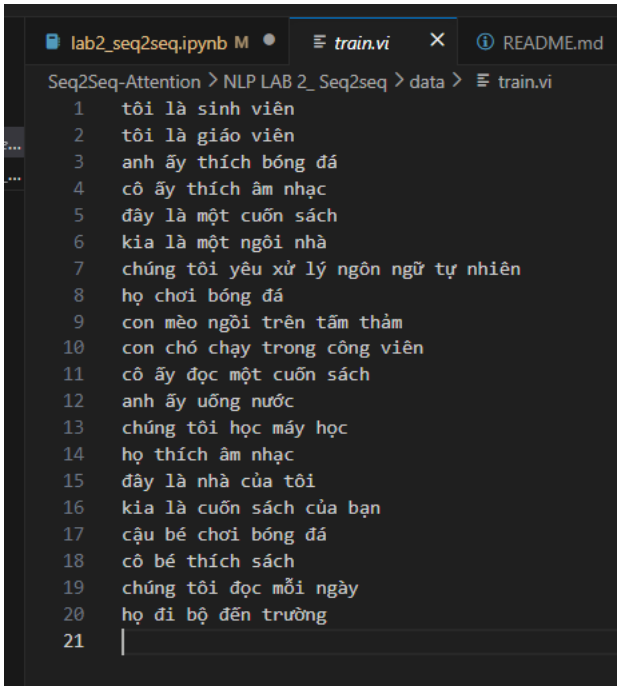
Pic 2. Data dev.vi.

- train.en: Dữ liệu huấn luyện tiếng Anh.



Pic 3. Data train.en.

- train.vi: Dữ liệu huấn luyện tiếng Việt.



Pic 4. Data train.vi.

➔ **Đánh giá dữ liệu:** Đơn giản, ít từ vựng khác nhau, chỉ dùng với mục đích học hiểu mô hình.

Phân tích code

Các cell có sẵn:

- Cell 1: Gọi các thư viện cần thiết cho bộ khung của mô hình Seq2Seq.

```
In [ ]:
import random
from pathlib import Path

import torch
import torch.nn as nn
import torch.nn.functional as F

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", DEVICE)
```

Pic 5. Cell code lấy các thư viện.

- Cell 2: Load đồng loạt nhiều dữ liệu.

```
In [ ]:
def load_lines(path):
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip()]

train_src = load_lines("data/train.en")
train_tgt = load_lines("data/train.vi")
dev_src = load_lines("data/dev.en")
dev_tgt = load_lines("data/dev.vi")

print(train_src[:3])
print(train_tgt[:3])
```

Pic 6. Cell code lấy các dữ liệu đồng thời.

- Cell 3: Gọi các token đặc biệt và tạo lại các hàm tiền xử lý của lab 1.

```
In [ ]:
SPECIAL_TOKENS = ["", "", "", ""]

class Vocab:
    def __init__(self):
        self.word2id = {}
        self.id2word = {}
        for token in SPECIAL_TOKENS:
            self.add_word(token)

    def add_word(self, word):
        if word not in self.word2id:
            idx = len(self.word2id)
            self.word2id[word] = idx
            self.id2word[idx] = word

    def build(self, sentences):
        # TODO: add words from tokenized sentences
        pass

    def encode(self, sentence):
        # TODO: return list of ids with  and
        pass

    def decode(self, ids):
        words = []
        for idx in ids:
            token = self.id2word.get(int(idx), "")
            if token == "":
                break
            if token not in ["", ""]:
                words.append(token)
        return " ".join(words)

def tokenize(sentence):
    return sentence.lower().split()
```

Pic 7. Sử dụng lại các hàm tiền xử lý từ lab 1.

- Cell 4: Thực hiện thêm vào các token đệm để tất các các câu dữ liệu có cùng độ dài.

```
In [ ]:
def pad_sequences(sequences, pad_id):
    max_len = max(len(seq) for seq in sequences)
    padded = []
    for seq in sequences:
        padded.append(seq + [pad_id] * (max_len - len(seq)))
    return torch.tensor(padded, dtype=torch.long)

# TODO:
# src_vocab = ...
# tgt_vocab = ...
# train_src_ids = ...
# train_tgt_ids = ...
```

Pic 8. Cell hàm gán token đệm.

- Cell 5: Xây dựng bộ Encoder cấu trúc khung của mô hình GRU.

```
In [ ]:
class Encoder(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, pad_id=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.rnn = nn.GRU(emb_dim, hidden_dim, batch_first=True)

    def forward(self, x):
        # TODO
        pass
```

Pic 9. Cell tạo hàm Encoder.

- Cell 6: Xây dựng hàm Decoder cấu trúc khung của mô hình GRU.

```
In [ ]:
class Decoder(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, pad_id=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.rnn = nn.GRU(emb_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, token, hidden):
        # TODO
        pass
```

Pic 10. Cell tạo hàm Decoder.

- Cell 7: Xây dựng mô hình Seq2Seq dựa vào mô hình GRU.

```
In [ ]:
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, bos_id, eos_id):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.bos_id = bos_id
        self.eos_id = eos_id

    def forward(self, src, tgt, teacher_forcing_ratio=0.5):
        # TODO
        pass

    @torch.no_grad()
    def greedy_decode(self, src, max_len=20):
        # TODO
        pass
```

Pic 11. Cell tạo hàm Seq2Seq.

- Cell 8: Huấn luyện mô hình với các siêu tham số có sẵn.

```
In [ ]:
# Suggested hyperparameters
EMB_DIM = 64
HIDDEN_DIM = 128
EPOCHS = 100
LR = 1e-3

# TODO:
# 1. create model
# 2. define optimizer and loss
# 3. run training loop
# 4. print loss every few epochs
```

Pic 12. Cell huấn luyện mô hình.

- Cell 9: Đánh giá trên bộ dữ liệu dev.

```
In [ ]:
# TODO:
# 1. encode dev set
# 2. run greedy decoding
# 3. print source / reference / prediction
```

Pic 13. Cell chỉ ra các việc cần làm để thực hiện đánh giá.

Thực hành code

- Công việc 1: Gọi tên các token đặc biệt của lab 1 đã có. Thêm hàm lưu trữ thư viện từ sang token và hàm encoder để mã hóa các từ được đưa vào từ input.

```python
SPECIAL_TOKENS = ["<PAD>", "<BOS>", "<EOS>", "<UNK>"]
```

Pic 14. Các token đặc biệt lấy từ lab 1.

```python
def build(self, sentences):
    """Build vocabulary from list of sentences"""
    for sentence in sentences:
        tokens = tokenize(sentence)
        for token in tokens:
            self.add_word(token)
    return self

def encode(self, sentence):
    """Encode sentence to list of IDs with <BOS> and <EOS>"""
    tokens = tokenize(sentence)
    ids = [self.word2id["<BOS>"]]
    for token in tokens:
        ids.append(self.word2id.get(token, self.word2id["<UNK>"]))
    ids.append(self.word2id["<EOS>"])
    return ids
```

Pic 15. 2 bộ hàm cần thiết trong TODO.

Câu hỏi phụ: Tại sao chúng ta lại cần các token phụ như <BOS> và <EOS> trong việc tạo ra thứ tự?

→ Trả lời: Đối với <BOS>, đây là token đầu tiên cho decoder ở bước thời gian t=0, để lại dấu cho model biết đang sinh chuỗi mới. Đối với <EOS>, đây là điều kiện dừng báo hiệu khi nào kết thúc việc sinh chuỗi mới trong quá trình Greedy Decoding.

- Công việc 2:

```python
# Hàm gán thêm các đệm để các chuỗi về cùng độ dài
def pad_sequences(sequences, pad_id):
    max_len = max(len(seq) for seq in sequences)
    padded = []
    for seq in sequences:
        padded.append(seq + [pad_id] * (max_len - len(seq)))
    return torch.tensor(padded, dtype=torch.long)

# Build vocabularies
src_vocab = Vocab().build(train_src)
tgt_vocab = Vocab().build(train_tgt)
# Encode training data
train_src_ids = [src_vocab.encode(sent) for sent in train_src]
train_tgt_ids = [tgt_vocab.encode(sent) for sent in train_tgt]

# Sử dụng hàm pad_sequences để gán các token đệm
PAD_ID = src_vocab.word2id["<PAD>"]
# Lấy 3 câu đầu tiên để minh họa
sample_src_ids = train_src_ids[:3]
print("Trước khi padding:")
for i, seq in enumerate(sample_src_ids):
    print(f"  Câu {i+1}: {seq} (độ dài: {len(seq)})")

# Áp dụng padding
padded_sample = pad_sequences(sample_src_ids, PAD_ID)
print(f"\nSau khi padding (PAD_ID={PAD_ID}):")
print(padded_sample)
print(f"Shape: {padded_sample.shape}")
```

[9] ✓ 0.0s

Pic 16. Thử nghiệm bộ encoding và đệm dữ liệu.

→ Kết quả thực hiện code:

```
Trước khi padding:
  Câu 1: [1, 4, 5, 6, 7, 2] (độ dài: 6)
  Câu 2: [1, 4, 5, 6, 8, 2] (độ dài: 6)
  Câu 3: [1, 9, 10, 11, 2] (độ dài: 5)

Sau khi padding (PAD_ID=0):
tensor([[ 1,  4,  5,  6,  7,  2],
        [ 1,  4,  5,  6,  8,  2],
        [ 1,  9, 10, 11,  2,  0]])
Shape: torch.Size([3, 6])
```

Pic 17. Kết quả đệm thêm token của dữ liệu câu ban đầu.

- Công việc 3, 4, 5: Xây dựng bộ encoder, decoder, Seq2Seq với Teaching Forcing. Đã có trong file seq2seq_model.py.

```python
class Encoder(nn.Module):
    def __init__(self, vocab_size: int, emb_dim: int, hidden_dim: int, pad_id: int = 0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.rnn = nn.GRU(emb_dim, hidden_dim, batch_first=True)

    def forward(self, x: torch.Tensor):
        emb = self.embedding(x)
        outputs, hidden = self.rnn(emb)
        return outputs, hidden


class Decoder(nn.Module):
    def __init__(self, vocab_size: int, emb_dim: int, hidden_dim: int, pad_id: int = 0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_dim, padding_idx=pad_id)
        self.rnn = nn.GRU(emb_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, token: torch.Tensor, hidden: torch.Tensor):
        emb = self.embedding(token)
        output, hidden = self.rnn(emb, hidden)
        logits = self.fc(output)
        return logits, hidden
```

Pic 18. Mô hình Encoder và Decoder vận dụng trên kiến trúc GRU.

```python
class Seq2Seq(nn.Module):
    def __init__(self, encoder: Encoder, decoder: Decoder, bos_id: int, eos_id: int, device: str = "cpu"):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.bos_id = bos_id
        self.eos_id = eos_id
        self.device = device

    def forward(self, src: torch.Tensor, tgt: torch.Tensor, teacher_forcing_ratio: float = 0.5):
        batch_size, tgt_len = tgt.size()
        vocab_size = self.decoder.fc.out_features

        outputs = torch.zeros(batch_size, tgt_len, vocab_size, device=src.device)

        _, hidden = self.encoder(src)
        decoder_input = tgt[:, 0].unsqueeze(1)

        for t in range(1, tgt_len):
            logits, hidden = self.decoder(decoder_input, hidden)
            outputs[:, t:t+1, :] = logits

            top1 = logits.argmax(dim=-1)
            use_teacher = random.random() < teacher_forcing_ratio
            decoder_input = tgt[:, t].unsqueeze(1) if use_teacher else top1

        return outputs
```

Pic 19. Mô hình Seq2Seq dựa trên kiến trúc GRU.

```python
@torch.no_grad()
def greedy_decode(self, src: torch.Tensor, max_len: int = 20):
    batch_size = src.size(0)
    _, hidden = self.encoder(src)
    decoder_input = torch.full((batch_size, 1), self.bos_id, dtype=torch.long, device=src.device)

    predictions = [[self.bos_id] for _ in range(batch_size)]

    for _ in range(max_len):
        logits, hidden = self.decoder(decoder_input, hidden)
        next_token = logits.argmax(dim=-1)
        decoder_input = next_token

        for i in range(batch_size):
            token_id = int(next_token[i, 0].item())
            predictions[i].append(token_id)

    return predictions
```

Pic 20. Bộ geedy decode để dự đoán các từ có xác suất cao nhất để lấy.

- Công việc 6: Huấn luyện mô hình

Định nghĩa optimizer và loss

```python
encoder = Encoder(len(src_vocab.word2id), EMB_DIM, HIDDEN_DIM, pad_id=PAD_ID)
decoder = Decoder(len(tgt_vocab.word2id), EMB_DIM, HIDDEN_DIM, pad_id=PAD_ID)
model = Seq2Seq(encoder, decoder, BOS_ID, EOS_ID, device=DEVICE).to(DEVICE)

print(f"Model created with {sum(p.numel() for p in model.parameters())} parameters")

# Define optimizer and loss
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_ID)
```

Pic 21. Xác định hàm optimizer và loss bằng thư viện torch

```python
# Training loop
for epoch in range(1, EPOCHS + 1):
    model.train()
    total_loss = 0

    # Simple batch training (process all data as one batch for simplicity)
    src_tensor = pad_sequences(train_src_ids, PAD_ID).to(DEVICE)
    tgt_tensor = pad_sequences(train_tgt_ids, PAD_ID).to(DEVICE)

    optimizer.zero_grad()

    # Forward pass
    output = model(src_tensor, tgt_tensor, teacher_forcing_ratio=0.5)

    # Compute loss (ignore first token which is <BOS>)
    output_dim = output.shape[-1]
    output = output[:, 1:, :].reshape(-1, output_dim)
    tgt = tgt_tensor[:, 1:].reshape(-1)

    loss = criterion(output, tgt)

    # Backward pass
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

    total_loss = loss.item()

    # Print progress
    if epoch % 10 == 0:
        print(f"Epoch {epoch}/{EPOCHS}, Loss: {total_loss:.4f}")

print("\nTraining completed!")
```

Pic 22. Tạo vòng lặp huấn luyện theo các epoch.

- Công việc 7: Đánh giá mô hình trên bộ dữ liệu dev sau khi huấn luyện

```python
# Encode dev set
dev_src_ids = [src_vocab.encode(sent) for sent in dev_src]
dev_src_tensor = pad_sequences(dev_src_ids, PAD_ID).to(DEVICE)

# Run greedy decoding
model.eval()
predictions = model.greedy_decode(dev_src_tensor, max_len=20)

# Print source / reference / prediction
print("=" * 80)
print("EVALUATION ON DEV SET")
print("=" * 80)

for i in range(len(dev_src)):
    src_text = dev_src[i]
    ref_text = dev_tgt[i]
    pred_ids = predictions[i]
    pred_text = tgt_vocab.decode(pred_ids)

    print(f"\n[Example {i+1}]")
    print(f"Source:     {src_text}")
    print(f"Reference:  {ref_text}")
    print(f"Prediction: {pred_text}")
    print("-" * 80)

[6]
```

Pic 23. Tạo cell thực hiện đánh giá theo dữ liệu dev.

4. Kết quả thực hiện

Kết quả quá trình huấn luyện với bộ dữ liệu mẫu:

```
Model created with 163835 parameters
Epoch 10/100, Loss: 3.7242
Epoch 20/100, Loss: 3.1668
Epoch 30/100, Loss: 2.6908
Epoch 40/100, Loss: 2.1014
Epoch 50/100, Loss: 1.7875
Epoch 60/100, Loss: 1.2419
Epoch 70/100, Loss: 1.1451
Epoch 80/100, Loss: 0.6942
Epoch 90/100, Loss: 0.5316
Epoch 100/100, Loss: 0.3581

Training completed!
```

Pic 24. Kết quả training mỗi epoch.

→ Đánh giá: Hàm loss của quá trình giảm rất nhanh chứng tỏ mô hình học hỏi rất nhanh do bộ dữ liệu ít, đồng thời với đó là mô hình có khả năng bị overfitting rất nhanh do những phần học hỏi rất ít và đã nắm được cách giải quyết đơn giản.

Kết quả so sánh trên bộ dữ liệu dev:

```
================================================================================
EVALUATION ON DEV SET
================================================================================

[Example 1]
Source:     i am a teacher
Reference:  tôi là giáo viên
Prediction: tôi là giáo viên
--------------------------------------------------------------------------------

[Example 2]
Source:     she likes music
Reference:  cô ấy thích âm nhạc
Prediction: cô ấy thích âm nhạc
--------------------------------------------------------------------------------

[Example 3]
Source:     this is a book
Reference:  đây là một cuốn sách
Prediction: đây là một cuốn sách
--------------------------------------------------------------------------------

[Example 4]
Source:     we study machine learning
Reference:  chúng tôi học máy học
Prediction: chúng tôi học máy học
--------------------------------------------------------------------------------
```

Pic 25. Bộ dữ liệu đánh giá trên bộ dev.

Đánh giá: Mô hình dễ dàng hiểu được các ý nghĩa trong bộ dữ liệu huấn luyện mẫu.

5. Giá trị rút ra

a) Câu hỏi 1.

Vậy với mô hình cơ bản Seq2Seq học hỏi tốt như vậy tại sao lại bị gặp khó khăn với những câu dài?

→ Trả lời: Mô hình Seq2Seq cơ bản (dựa trên kiến trúc mạng nơ-ron hồi quy như RNN, LSTM hoặc GRU) thường gặp hiện tượng suy giảm hiệu suất nghiêm trọng khi xử lý các chuỗi văn bản dài. Hạn chế này xuất phát từ hai nguyên nhân cốt lõi về mặt kiến trúc:

- Information Bottleneck: Trong kiến trúc Seq2Seq tiêu chuẩn, Encoder có nhiệm vụ đọc toàn bộ chuỗi đầu vào và nén tất cả thông tin ngữ nghĩa, ngữ pháp vào một vector ngữ cảnh (context vector) có kích thước cố định (chính là trạng thái ẩn cuối cùng của Encoder). Khi câu quá dài, một vector với số chiều cố định (ví dụ: hidden_dim = 256 hoặc 512) không đủ sức chứa để biểu diễn trọn vẹn mọi chi tiết. Điều này dẫn đến việc mất mát thông tin (information loss), đặc biệt là các thông tin xuất hiện ở đầu câu thường bị "ghi đè" bởi các từ xuất hiện sau.
- Hạn chế trong việc học phụ thuộc xa (Long-term Dependencies): Mặc dù các cổng (gates) trong LSTM hay GRU được thiết kế để giảm thiểu vấn đề suy giảm đạo hàm (vanishing gradient), chúng vẫn gặp khó khăn khi phải duy trì luồng thông tin qua quá nhiều bước thời gian. Khi Decoder tiến hành sinh văn bản ở những bước

cuối cùng của một câu dài, nó thường có xu hướng "quên" mất những ngữ cảnh quan trọng ban đầu, dẫn đến việc sinh ra các từ sai ngữ pháp hoặc lệch ý nghĩa.

b) Câu hỏi 2.

Câu hỏi phụ: Teaching Forcing là gì? Tại sao nó lại cần thiết trong huấn luyện mô hình?

→ Trả lời: Teacher Forcing (Ép buộc qua giáo viên) là một kỹ thuật được sử dụng trong quá trình huấn luyện các mô hình ngôn ngữ hoặc mô hình sinh chuỗi tự hồi quy (như Seq2Seq với GRU/LSTM).

Trong cơ chế sinh văn bản thông thường, đầu vào của Decoder ở bước thời gian t chính là từ (token) mà nó vừa dự đoán ra ở bước t-1. Tuy nhiên, khi áp dụng Teacher Forcing, thay vì dùng dự đoán của chính mô hình, chúng ta trực tiếp sử dụng từ khóa thực tế (ground truth token) từ tập dữ liệu huấn luyện (chuỗi đích) để làm đầu vào cho bước tiếp theo.

Kỹ thuật này giải quyết những rào cản lớn trong giai đoạn đầu huấn luyện mô hình với hai lợi ích cốt lõi sau:

- Ngăn chặn hiệu ứng lan truyền lỗi (Cascading Errors): Khi mới bắt đầu huấn luyện, các trọng số của mô hình được khởi tạo ngẫu nhiên nên dự đoán thường rất thiếu chính xác. Nếu sử dụng một từ sai ở bước $t-1$ làm đầu vào cho bước t, toàn bộ phần còn lại của câu sẽ bị chệch hướng hoàn toàn. Teacher Forcing giúp "nắn" mô hình quay lại đúng ngữ cảnh ngay lập tức, bất kể trước đó nó dự đoán sai thế nào.
- Hội tụ nhanh hơn và ổn định hơn: Việc liên tục được cung cấp ngữ cảnh đúng đắn giúp hàm mất mát (loss function) cung cấp các tín hiệu cập nhật gradient (đạo hàm) chính xác hơn. Điều này giúp mô hình nhanh chóng học được cấu trúc ngôn ngữ và giảm thiểu đáng kể thời gian huấn luyện.

c) Câu hỏi 3.

Câu hỏi phụ: Phân phối xác suất gì đã được mô hình hóa bởi decoder ở thời gian bước t?

→ Trả lời: Ở thời gian bước t, Decoder trong kiến trúc Seq2Seq (sử dụng GRU) mô hình hóa phân phối xác suất có điều kiện (Conditional Probability Distribution) của từ (token) tiếp theo.

Về mặt thống kê, đây là một phân phối Categorical (phân phối rời rạc đa thức) trải rộng trên toàn bộ không gian từ vựng (vocabulary). Phân phối này được tính toán dựa trên

vector ngữ cảnh được mã hóa từ chuỗi đầu vào, kết hợp với trạng thái ẩn chứa lịch sử của các từ đã được sinh ra trước đó.

Biểu diễn toán học của phân phối này như sau:

$$P(y_t|y_1, y_2, \ldots, y_{t-1}, c)$$

Trong đó:

- y_t: Token mục tiêu cần dự đoán tại bước t.
- $y_1, y_2, \ldots, y_{t-1}$: Các token đã được sinh ra ở các bước thời gian trước đó.
- c: Vector ngữ cảnh (context vector) mang thông tin nén của toàn bộ chuỗi đầu vào từ Encoder.